EXE Group Management System

Design document

Course: PRN232 – Building Cross-Platform Back-End Application with .NET

Assignment: Final Assignment

Architecture: Microservices architecture, documented with the C4 model (Level 1 & Level 2)

Instructor: PhuongLHK

Semester: Fall 2026 (FA26)

Version: 3.2 · November 5, 2026

Version history

Version	Date	Changes
2.2	November 5, 2026	Versioned routes (/api/v1), refresh tokens and sign-out (Identity Service).
3.2	November 5, 2026	Lecturer feedback: FAP terms (Subject, Course = subject + class + semester), admin-led rooms instead of student-created groups, group types with major minimums, skills, deadline and finalize, group chat. Join requests move into the Group Service; new Recommendation Service (gRPC) and Realtime Service (SignalR).

Contents

1. Context 3
2. Problems 3
3. Solutions 3
4. Main Actors 4
5. Main Features 4
5.1 Features 4
5.2 Business rules 5
5.3 Statuses and codes 5
5.4 Grouping workflow 6
6. System Architecture (C4 model, Levels 1 and 2) 7
6.1 Level 1: System Context 8
6.2 Level 2: Containers 9
6.3 Communication 10
6.4 Key flows 10
6.5 Events (Redis stream group-events) 11
6.6 Background jobs 11
6.7 gRPC contract: Recommendation Service 11
6.8 Layered architecture and API conventions 12
7. Technology 13
8. ERD (conceptual) 13
9. Physical Database 14
9.1 IdentityDb (Identity Service) 15
9.2 GroupDb (Group Service) 17
9.3 ChatDb (Realtime Service) 24
Appendix A. REST API 26
Identity Service 26
Group Service: master data 26
Group Service: students and courses 26
Group Service: rooms and applications 27
Realtime Service 27
Appendix B. Assignment requirements 27

1. Context

At FPT University the EXE subjects (EXE101 Experiential Entrepreneurship 1 and EXE201) are taught to classes such as SE1919 every semester. In every class the students work in project groups of four to six members from at least two different majors, so that each start-up team covers technology, design and business. Forming the groups is the first task of the semester and blocks everything that follows: topic registration, checkpoints and grading.

Today the groups form informally through chat groups and spreadsheets, and the academic staff only see the result. The PRN232 lecturer's feedback describes the process the school wants instead: the staff prepare the semester, the subjects and the courses, add the students with their major, create rooms for each course from a simple formula and publish them. Students rate their own strengths and join rooms; the first student in a room leads it, may lock it and then reviews applications. When the deadline passes the staff arrange the remaining students and finalize the official groups, which everyone can then see.

The EXE Group Management System implements this process as a distributed .NET application. It is also the PRN232 Final Assignment, so it demonstrates REST APIs, background jobs, a message broker, a gRPC service and a containerized deployment.

2. Problems

ID	Problem	Impact
P1	Groups form informally	Chat groups and spreadsheets: the staff have no live view and find the students without a group too late.
P2	Major-mix rules are hard to keep by hand	Groups miss a required major (for example no marketing student) and have to be rebuilt.
P3	Nobody knows who is good at what	Students pick friends instead of complementary strengths.
P4	Deadline and final list are handled by hand	Late changes and disputes about who is in which group.
P5	Group communication is scattered	Leaders cannot reach applicants and members in one place.
P6	Lecturers cannot see their class's groups	They mentor groups they have no list of.

3. Solutions

ID	Solution	How	Solves
S1	Admin-led rooms	The admin generates the rooms of a course (rooms = ceil(students / MaxGroupSize x k), k = 1.2) and publishes them; students join rooms instead of creating groups.	P1
S2	Rules in the backend	One room per student per course, seats reserved for the majors a group type still needs, eligibility (minimum size and majors), deadline lock.	P2, P4
S3	Skills and suggestions	Students rate their strengths 1-5; a gRPC Recommendation Service ranks rooms for a student and placements for the admin.	P3
S4	Deadline and finalize	The course expires at ExpireAt (background job); the admin places the remaining students and finalizes numbered, read-only official groups.	P4
S5	Built-in chat and notifications	One chat per room and real-time notifications (Realtime Service, SignalR).	P5
S6	Lecturer view	Lecturer accounts with a read-only view of the courses they teach.	P6
S7	Distributed .NET platform	Microservices behind an API gateway, a database per service, transactional outbox and Redis Streams, Docker Compose, CI and a demo deployment.	Assignment

4. Main Actors

The Admin is the moderator described in the lecturer's feedback; in the system it is the Administrator role. A Room Leader is a student, not a separate account.

Actor	Role in the token	Description	Main permissions
Admin	Administrator	Academic staff who runs group formation: the moderator of the lecturer's feedback.	Accounts and roles; semesters, subjects, majors, skills, group types; courses and rosters; generate, publish and expire rooms; move and place students; finalize.
Lecturer	Lecturer	Teaches one or more EXE classes.	Read-only: progress, rooms and members, students without a room and the official groups of their courses.
Student	Student	EXE student enrolled in courses of a semester.	Rate skills; browse rooms and get suggestions; join, leave and apply; chat with the room; see the official groups.
Room Leader	Student (leader of a room)	First student to join a room, or the member the leader hands over to.	Set the topic; lock and unlock; approve and reject applications; remove a member with a reason; hand over leadership.
Scheduler	-	Background jobs of the services.	Publish and expire courses on time; publish and consume events; clean up.

5. Main Features

5.1 Features

ID	Feature	Actors	Service
F01	Sign in, refresh the token, sign out, edit own profile	All	Identity
F02	Manage accounts and roles: create, import students, lock and unlock	Admin	Identity
F03	Manage semesters, subjects, majors and skills	Admin (read: all)	Group
F04	Manage group types (minimum students per major, minimum different majors) and allow them per semester	Admin	Group
F05	Add students to a semester with their major; enroll them in courses	Admin	Group
F06	Create courses (subject + class + semester): lecturer, group sizes, k, PublishAt, ExpireAt	Admin	Group
F07	Generate the rooms of a course with their group type	Admin	Group
F08	Publish and expire a course, manually or on time; move the deadline later	Admin, Scheduler	Group
F09	Rate own skills 1-5	Student	Group
F10	Browse rooms with search, filters, sorting and paging; see members, majors, skills and topic	Student, Lecturer, Admin	Group
F11	Get room suggestions	Student	Group + Recommendation (gRPC)
F12	Join an open room; leave a room	Student	Group
F13	Apply to a locked room; withdraw the application	Student	Group
F14	Set the topic, lock and unlock, approve and reject, remove a member, hand over leadership	Room Leader	Group
F15	Course overview: students without a room, rooms that are not eligible	Admin, Lecturer	Group
F16	Move and place students after the deadline, with placement suggestions	Admin	Group + Recommendation (gRPC)
F17	Finalize the course into numbered, read-only official groups	Admin	Group
F18	Chat with the members of the room	Student	Realtime
F19	Receive notifications: applications, decisions, removals, phase changes	All	Realtime
F20	Scheduled work: publish and expire on time, publish events, clean up	Scheduler	Group, Realtime

5.2 Business rules

The Group Service enforces these rules in its Services layer; the database keeps the invariants it can check (keys, filtered unique indexes, checks).

ID	Rule
BR-01	A course is one subject taught to one class in one semester; the class code is unique per subject and semester.
BR-02	A student takes part in a semester with exactly one major (imported by the admin) and can be enrolled only in courses of that semester.
BR-03	Rooms of a course = ceil(enrolled students / MaxGroupSize x k), k = RoomFactor (default 1.2). Every room has Capacity = MaxGroupSize and a group type allowed in the semester.
BR-04	Rooms are generated, renamed or deleted only while the course is in Setup.
BR-05	Students join, leave and apply only while the course is Published.
BR-06	A student is in at most one room per course. Joining a room cancels their pending applications in that course (JOINED_ANOTHER_ROOM).
BR-07	The first student to join an empty room becomes its leader. When the leader leaves, the earliest remaining member takes over; the leader may also hand over leadership.
BR-08	Open room: enrolled students join directly. Locked room: students apply and the leader decides. Unlocking, or the room becoming empty, cancels its pending applications (ROOM_REOPENED).
BR-09	Every join or approval must keep enough free seats for the majors the group type still needs: the minimum per major and the minimum number of different majors (COMPOSITION_RULE).
BR-10	A full room accepts no one. When a room becomes full its pending applications are rejected (ROOM_FULL).
BR-11	Until the course expires the leader may remove a member and must give a reason; the removed student cannot join or apply to that room again (REMOVED_FROM_ROOM). Memberships are kept as history: Left, Removed, Moved.
BR-12	At ExpireAt, or when the admin expires the course, pending applications are rejected (GROUPING_EXPIRED) and students can no longer change rooms; only the admin moves or places students (JoinedVia = Admin, EndReason = Moved). While the course is Published the admin may move ExpireAt later.
BR-13	A room is eligible when it has at least MinGroupSize members and the majors its group type requires.
BR-14	Finalize needs every enrolled student in a room and every non-empty room eligible; the admin may confirm an exception (for example a class too small for the rules). Empty rooms are deleted; the others get GroupNo 1..n and become read-only official groups.
BR-15	Skills are self-rated 1-5 per semester. They are shown to students and leaders and used for suggestions; they never block an action.
BR-16	A lecturer sees only the courses they teach. Only the members of a room can read its chat.

5.3 Statuses and codes

Course phase (Courses.Phase)

Value	Meaning
Setup	The admin imports students, generates rooms and sets group types; students rate skills.
Published	Students join, leave and apply; leaders lock, review applications and remove members.
Expired	Students are frozen; the admin moves and places students.
Finalized	Rooms are numbered official groups; the list is read-only.

Room status (Rooms.Status)

Value	Meaning
Open	Enrolled students join directly.
Locked	Students apply; the leader approves or rejects.
Finalized	Official group (GroupNo set); read-only.

Membership (RoomMembers.JoinedVia / EndReason)

Value	Meaning
Open / Application / Admin	How the student entered: direct join, approved application, admin placement.
Left / Removed / Moved	How the membership ended: the student left, the leader removed them, the admin moved them.

Application (JoinRequests.Status)

Value	Meaning

Value	Meaning
Pending	Waiting for the leader.
Approved	The student became a member.
Rejected	By the leader or automatically (see ReasonCode).
Cancelled	Withdrawn by the student or cancelled automatically (see ReasonCode).

Reason codes stored on rejected or cancelled applications (JoinRequests.ReasonCode)

Code	Meaning
REJECTED_BY_LEADER	The room leader rejected the request (optional text in ReasonText).
WITHDRAWN	The student cancelled their own application.
ROOM_FULL	The room reached its capacity; its pending applications are rejected.
ROOM_REOPENED	The room was unlocked (by its leader, or it became empty); students join directly.
COMPOSITION_RULE	Adding the student would make the group type's major minimums impossible.
JOINED_ANOTHER_ROOM	Automatic cancellation: the student joined another room of the course.
GROUPING_EXPIRED	The course expired (by the admin or at ExpireAt); pending applications are rejected.

Business error codes returned with HTTP 409 (nothing is stored)

Code	Meaning
COURSE_NOT_PUBLISHED	Students join, leave and apply only while the course is Published.
NOT_ENROLLED	The student is not on the course roster.
ALREADY_IN_ROOM	The student is already a member of a room of the course.
ROOM_NOT_LOCKED	The room is Open: the student joins it directly.
ROOM_LOCKED	The room is Locked: the student must apply.
ROOM_FULL	The room has no free seat.
COMPOSITION_RULE	The join would make the group type's major minimums impossible.
REMOVED_FROM_ROOM	The leader removed the student from this room; they cannot join or apply to it again.
NOT_ROOM_LEADER	Only the room leader can do this.
COURSE_NOT_EXPIRED	The admin moves students only after the course expired.
FINALIZE_BLOCKED	Some students have no room, or some rooms are not eligible and no exception was accepted.

5.4 Grouping workflow

Figure A puts the phases of a course, the statuses of a room and the main rules on one page.

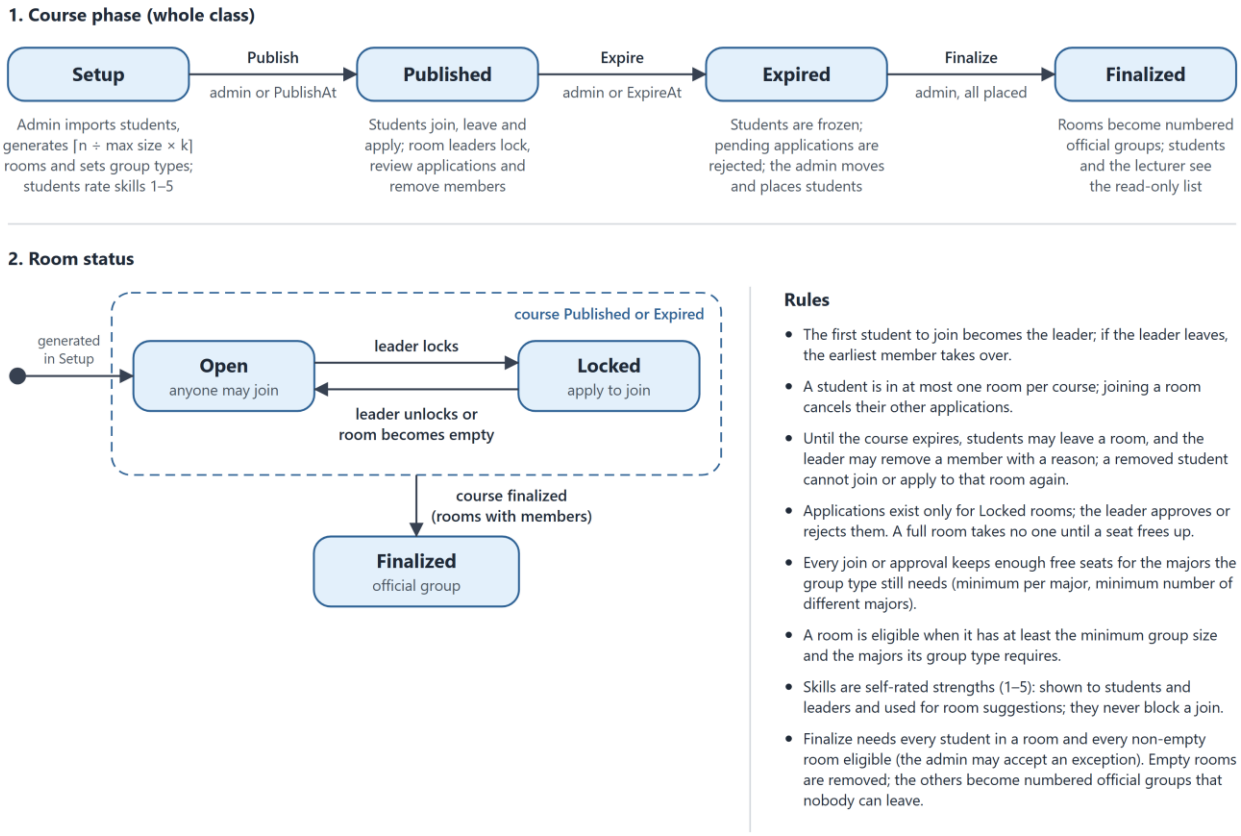


Figure A. Grouping workflow: course phases, room statuses and rules

6. System Architecture (C4 model, Levels 1 and 2)

The architecture follows the C4 model; this document uses Level 1 (System Context) and Level 2 (Container). The same model, with component and deployment views, is kept as code in docs/architecture/workspace.dsl.

6.1 Level 1: System Context

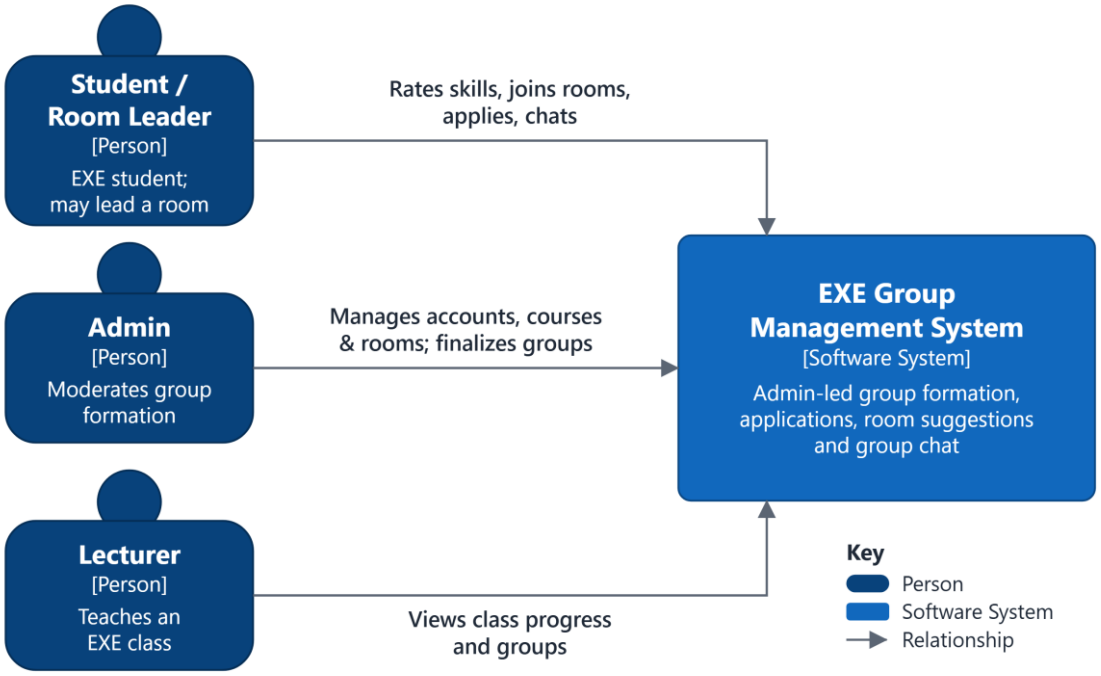


Figure 1. System Context diagram (C4 Level 1)

Three kinds of people use the system through the browser. There is no external system: accounts and rosters are imported by the admin, and notifications are shown in the application.

6.2 Level 2: Containers

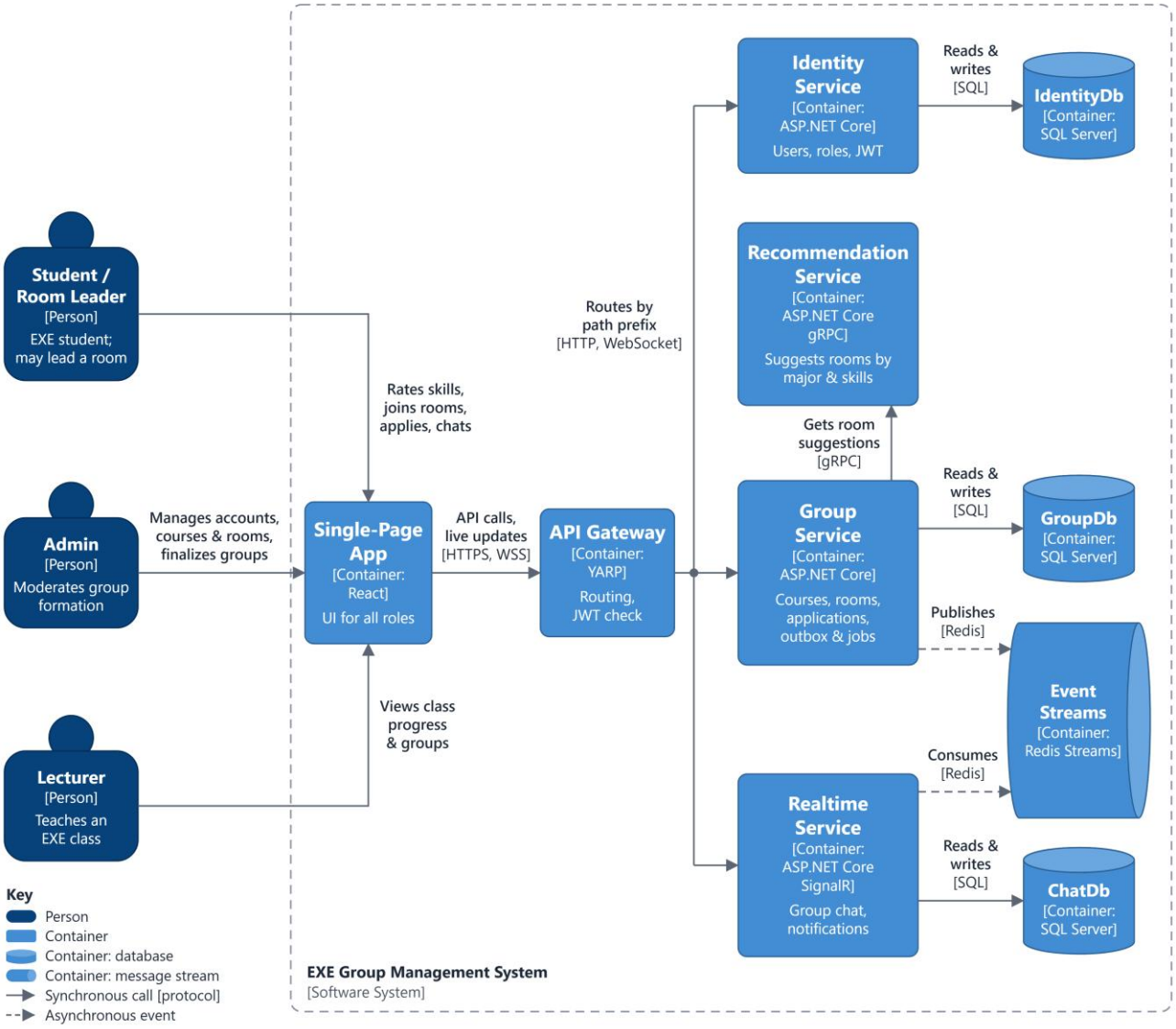


Figure 2. Container diagram (C4 Level 2)

Container	Technology	Responsibility	Port / data
Single-Page App	React 19, TypeScript, Vite, nginx	UI for all roles; nginx serves the app and proxies /api, /hubs and /swagger to the gateway.	web: 3000 -> 80
API Gateway	ASP.NET Core 10, YARP 2.3	Single entry point: JWT check at the edge, routing by path prefix, WebSocket pass-through, sign-in rate limit, merged Swagger UI.	8080
Identity Service	ASP.NET Core 10 Web API	Sign-in, JWT access and refresh tokens, users and roles.	IdentityDb
Group Service	ASP.NET Core 10 Web API	Semesters, subjects, majors, skills, group types, courses and rosters, rooms, memberships, applications and finalize; calls the Recommendation Service; writes the outbox; schedules publish and expiry.	GroupDb
Recommendation Service	ASP.NET Core 10 gRPC	Ranks rooms for a student and placements for students without a room from majors and skills. Stateless, no database, not routed by the gateway.	8081 (HTTP/2)
Realtime Service	ASP.NET Core 10 Web API + SignalR	Group chat and notifications; consumes the group events and pushes them to browsers.	ChatDb, /hubs
IdentityDb, GroupDb, ChatDb	SQL Server 2022	One database per service; no cross-database foreign keys.	1433
Event Streams	Redis 7 Streams	Stream group-events; consumer group realtime.	6379

6.3 Communication

Style	Between	Protocol	Used for
REST (synchronous)	Browser -> Gateway -> services	HTTPS, JSON envelope, /api/v1, JWT	Every user action.
gRPC (synchronous)	Group Service -> Recommendation Service	Protobuf over HTTP/2 (:8081), 2 s deadline	Room suggestions and placement suggestions; the Group Service falls back to unranked lists.
Events (asynchronous)	Group Service -> Redis Streams -> Realtime Service	XADD from the outbox; XREADGROUP and XACK	Notifications, chat membership, live room board.
WebSocket	Browser <-> Realtime Service (through the gateway)	SignalR hub /hubs/realtime	Chat messages, notifications and room changes pushed to the browser.

Rule of thumb: the browser calls REST; a service that needs an answer now calls gRPC; a service that only announces that something happened publishes an event.

6.4 Key flows

Apply to a locked room and approval (single transaction)

1. The student calls POST /api/v1/join-requests {roomId, message}. The Group Service checks BR-05, BR-06, BR-08 and BR-11, stores the application as Pending and writes JoinRequestSubmitted to the outbox.
2. The OutboxPublisher adds the event to group-events; the Realtime Service notifies the leader.
3. The leader calls POST /api/v1/join-requests/{id}/approve. In one transaction the Group Service checks the seat and the majors again (BR-09, BR-10), inserts the membership (JoinedVia = Application), updates Rooms.MemberCount (RowVersion), cancels the student's other pending applications, rejects the room's pending applications if it is now full and writes the events.
4. The Realtime Service notifies the applicant, adds them to the room chat and refreshes the room board.

Room suggestions (REST -> gRPC)

1. The student calls GET /api/v1/courses/{id}/rooms/suggestions?top=3.
2. The Group Service loads the student's major and skills, the rooms with a free seat, their members' majors and skills and the group type minimums.
3. It calls Recommendation.SuggestRooms with a 2 s deadline.
4. The Recommendation Service drops the rooms the student cannot join, scores the others and returns them ranked, with reasons.
5. If the call fails the Group Service returns the rooms without ranking.

Deadline (background job)

1. Every minute the PhaseScheduler finds Published courses whose ExpireAt has passed.
2. In one transaction it sets Phase = Expired, rejects the pending applications (GROUPING_EXPIRED) and writes CoursePhaseChanged.
3. The Realtime Service notifies the students of the course and the admin.

Finalize

1. The admin opens GET /api/v1/courses/{id}/overview: students without a room and rooms that are not eligible.
2. The admin places students with POST /api/v1/courses/{id}/placements, helped by GET /api/v1/courses/{id}/placement-suggestions (gRPC).
3. POST /api/v1/courses/{id}/finalize {acceptExceptions} checks BR-14, deletes empty rooms, numbers the others, sets them Finalized and writes CoursePhaseChanged.

4. Students and the lecturer are notified and see the official list.

6.5 Events (Redis stream group-events)

The Group Service writes every event to OutboxEvents in the same transaction as the change; the OutboxPublisher adds it to the stream. Every payload also carries eventId, occurredAt and correlationId.

Event	Produced when	Payload	Realtime Service reaction
CoursePhaseChanged	Publish, expire, finalize	courseId, phase	Notify the enrolled students (and the lecturer when finalized).
RoomMemberJoined	Join, approval, admin placement	courseId, roomId, studentId, joinedVia	Add the student to the room chat (create it for the first member); notify the leader; refresh the board.
RoomMemberLeft	Leave, removal, admin move	courseId, roomId, studentId, endReason, endNote	Remove the student from the chat; notify a removed or moved student; refresh the board.
RoomLeaderChanged	First join, leader left, hand-over	roomId, leaderId	Notify the new leader.
RoomStatusChanged	Lock, unlock, finalize	roomId, status	Refresh the board.
JoinRequestSubmitted	Application stored	requestId, roomId, studentId, leaderId	Notify the leader.
JoinRequestDecided	Approve, reject, automatic rejection or cancellation	requestId, roomId, studentId, status, reasonCode	Notify the applicant.

6.6 Background jobs

Job	Service	Schedule	What it does
OutboxPublisher	Group	Every 2 s	XADD unpublished OutboxEvents to group-events and set PublishedAtUtc; RetryCount and LastError on failure.
PhaseScheduler	Group	Every 1 min	Publish courses at PublishAt; expire courses at ExpireAt (BR-12).
OutboxCleanup	Group	Daily	Delete published events older than 7 days.
GroupEventsConsumer	Realtime	Continuous	XREADGROUP (group realtime); skip ids found in ProcessedEvents; apply; XACK after commit; XAUTOCLAIM stale entries.
ProcessedEventsCleanup	Realtime	Daily	Delete processed event ids older than 7 days.

6.7 gRPC contract: Recommendation Service

The Recommendation Service is the system's independent gRPC service. It owns no data: the Group Service sends the student, the rooms and the rules in each call (src/Contracts, recommendation.proto).

```
syntax = "proto3";

option csharp_namespace = "ExeGroup.Contracts.Recommendation";

package exegroup.recommendation.v1;

service Recommendation {
  rpc SuggestRooms (SuggestRoomsRequest) returns (SuggestRoomsReply);
  rpc SuggestPlacements (SuggestPlacementsRequest) returns (SuggestPlacementsReply);
}

message Student {
  string student_id = 1;
  string major_code = 2;
  map<string, int32> skills = 3;
}

message Room {
  string room_id = 1;
  string name = 2;
  int32 capacity = 3;
  repeated Student members = 4;
```

```
  map<string, int32> min_per_major = 5;
  int32 min_distinct_majors = 6;
  int32 min_group_size = 7;
}

message Reason {
  string code = 1;
  string detail = 2;
}

message SuggestRoomsRequest {
  Student student = 1;
  repeated Room rooms = 2;
  int32 top = 3;
}

message RoomSuggestion {
  string room_id = 1;
  double score = 2;
  repeated Reason reasons = 3;
}

message SuggestRoomsReply {
  repeated RoomSuggestion suggestions = 1;
}

message SuggestPlacementsRequest {
  repeated Student unassigned = 1;
  repeated Room rooms = 2;
}

message Placement {
  string student_id = 1;
  string room_id = 2;
  repeated Reason reasons = 3;
}

message SuggestPlacementsReply {
  repeated Placement placements = 1;
  repeated string unplaced_student_ids = 2;
}
```

Scoring

- Rooms the student cannot join are dropped: full rooms and rooms where the join would break BR-09.
- MAJOR_NEEDED: the room still needs the student's major (minimum per major or a new distinct major).
- SKILL_GAP: for every skill, the student's level above the best level in the room (detail = skill code).
- BELOW_MINIMUM: the room has fewer than MinGroupSize members.
- Placements: students with the scarcest majors are placed first, each into the best-scoring room that keeps BR-09; students that fit nowhere are returned as unplaced.

6.8 Layered architecture and API conventions

- Three projects per service: API -> Services -> Repositories. Architecture tests fail the build when the API uses EF Core, when Services or Repositories use ASP.NET Core, or when a service references another one.
- Four model types: Entity (Repositories), business model (Services), Request and Response (API).
- Every response is an envelope {success, message, data, errors, traceId, pagination}; failures are coded exceptions mapped to 400, 401, 403, 404 and 409 with a business code (the codes of section 5.3).
- Versioned routes /api/v{version}/<plural-resource>; authentication is required unless an endpoint is marked anonymous; roles through [Authorize(Roles = ...)], room leadership and lecturer ownership checked in the Services layer.
- List endpoints accept search, sort (several fields, '-' for descending), page, pageSize (default 20, at most 100), fields and expand, plus their own filters.

- Serilog with a correlation id (X-Correlation-ID) carried across REST, gRPC and events; health checks per service; one OpenAPI document per service, merged at the gateway.

7. Technology

Area	Technology
Runtime	.NET 10 (SDK pinned in global.json), C# latest, nullable enabled, warnings as errors
Backend	ASP.NET Core 10; three projects per service (API -> Services -> Repositories)
Data	Entity Framework Core 10, SQL Server 2022, code-first migrations matching database/schema.sql
Gateway	YARP 2.3
Synchronous service calls	gRPC: Grpc.AspNetCore / Grpc.Net.ClientFactory 2.85, Protobuf over HTTP/2
Messaging	Redis 7 Streams (StackExchange.Redis 3.x), transactional outbox, idempotent consumer
Real time	ASP.NET Core SignalR (chat, notifications, live room board)
Background jobs	BackgroundService + PeriodicTimer through a shared PeriodicBackgroundService base
Security	JWT Bearer HS256, rotating refresh tokens, ASP.NET Core PasswordHasher
API surface	URL versioning (Asp.Versioning 10), JSON and XML, DataAnnotations + FluentValidation
API documentation	Swashbuckle 10 (OpenAPI), one document per service, merged Swagger UI at the gateway
Logging	Serilog with correlation id; request-logging and exception middleware
Frontend	React 19, TypeScript, Vite, React Router 7, TanStack Query 5, nginx
Tests	xUnit v3, Testcontainers (real SQL Server), NetArchTest
Deployment	Docker, Docker Compose (8 containers), GitHub Actions CI and demo deployment

8. ERD (conceptual)

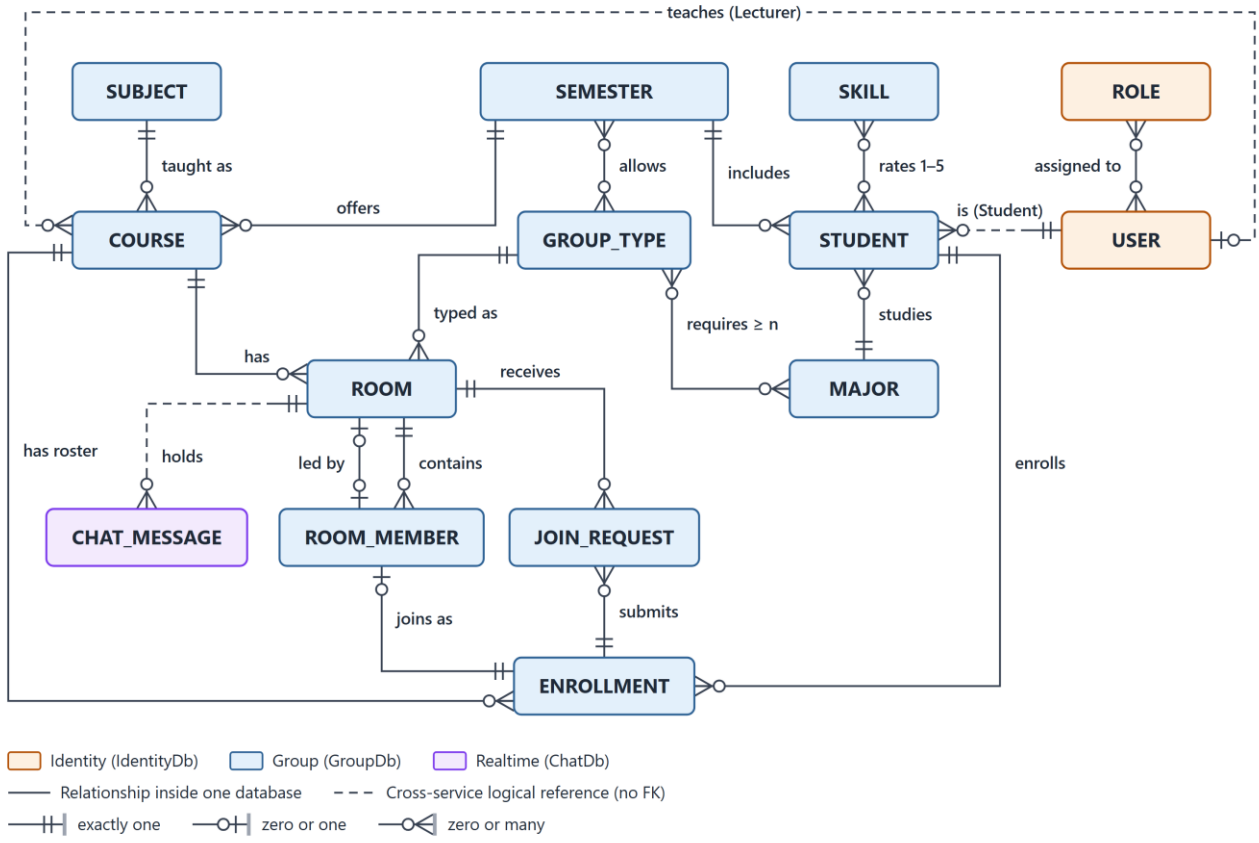


Figure 3. Conceptual ERD

Colours show the database that owns an entity; dashed lines are logical references across services (no foreign key).

Entity	Meaning	Database
USER, ROLE	Accounts and their global roles.	IdentityDb
SEMESTER	Academic term, e.g. FA26.	GroupDb
SUBJECT	Subject such as EXE101.	GroupDb
COURSE	A subject taught to one class in one semester, with grouping settings, phase and lecturer.	GroupDb
STUDENT	A user taking part in a semester, with one major.	GroupDb
MAJOR, SKILL	Study majors; strengths students rate (rates 1-5).	GroupDb
ENROLLMENT	A student on the roster of a course.	GroupDb
GROUP_TYPE	Major mix rules (requires >= n of a major); allowed per semester.	GroupDb
ROOM	A slot of a course; becomes an official group at finalize.	GroupDb
ROOM_MEMBER	A membership of an enrolled student in a room, current or ended; the room is led by one.	GroupDb
JOIN_REQUEST	An application of an enrolled student to a locked room.	GroupDb
CHAT_MESSAGE	A message in the room's chat.	ChatDb

9. Physical Database

One SQL Server database per service. database/schema.sql is generated from the same design data as these figures, and the EF Core migrations must produce the same tables, keys, indexes and checks. Key column: PK primary key, FK foreign key, UK member of a unique key, LR logical reference to another database (no FK). Filtered unique indexes (WHERE ...) cover only the rows that match.

9.1 IdentityDb (Identity Service)

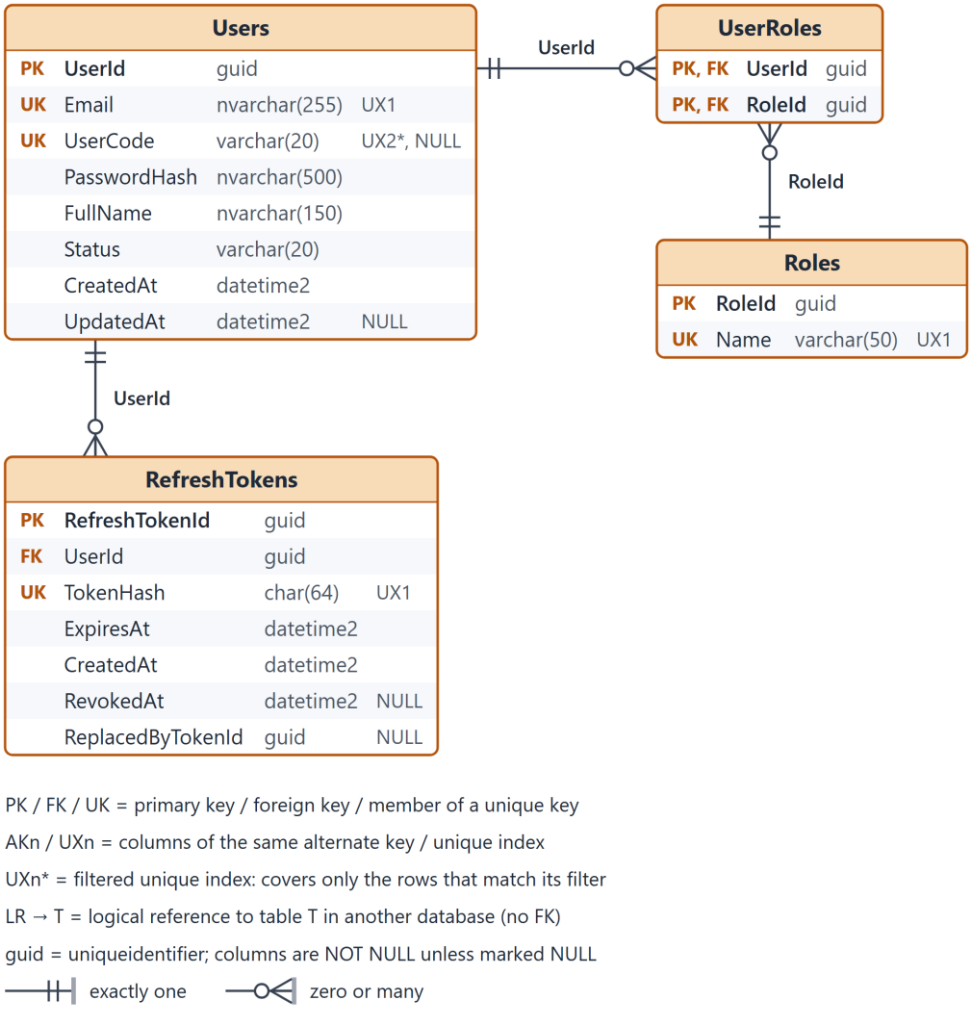


Figure 4. Physical database: IdentityDb

Users

User accounts for all roles.

Column	Type	Null	Key	Default	Description
UserId	uniqueidentifier		PK		Surrogate key (sequential GUID generated by EF Core).
Email	nvarchar(255)		UK		University e-mail used to sign in.
UserCode	varchar(20)	NULL	UK		Student code (e.g. SE180001) or staff code; required for Student accounts; unique when present.
PasswordHash	nvarchar(500)				Salted PBKDF2 hash (ASP.NET Core PasswordHasher).
FullName	nvarchar(150)				Display name.
Status	varchar(20)			'Active'	Account state: Active, Locked.
CreatedAt	datetime2			SYSUTCDATETIME()	Creation time (UTC).
UpdatedAt	datetime2	NULL			Last update time (UTC).

Keys: PK_Users (UserId)

Unique indexes: UX_Users_Email (Email); UX_Users_UserCode (UserCode) WHERE [UserCode] IS NOT NULL

Checks: CK_Users_Status: [Status] IN ('Active', 'Locked')

Roles

Global roles (seeded): Student, Lecturer, Administrator (the Admin actor).

Column	Type	Null	Key	Default	Description
RoleId	uniqueidentifier		PK		Surrogate key (fixed seed values).
Name	varchar(50)		UK		Role name: Student, Lecturer, Administrator.

Keys: PK_Roles (RoleId)

Unique indexes: UX_Roles_Name (Name)

Seed rows: Student, Lecturer, Administrator

UserRoles

Many-to-many assignment of roles to users.

Column	Type	Null	Key	Default	Description
UserId	uniqueidentifier		PK, FK		References Users(UserId); cascade delete.
RoleId	uniqueidentifier		PK, FK		References Roles(RoleId).

Keys: PK_UserRoles (UserId, RoleId)

Indexes: IX_UserRoles_RoleId (RoleId)

Foreign keys: FK_UserRoles_Users (UserId) -> Users (UserId), ON DELETE CASCADE; FK_UserRoles_Roles (RoleId) -> Roles (RoleId), ON DELETE NO ACTION

RefreshTokens

Rotating refresh tokens; only a hash is stored.

Column	Type	Null	Key	Default	Description
RefreshTokenId	uniqueidentifier		PK		Surrogate key.
UserId	uniqueidentifier		FK		References Users(UserId); cascade delete.
TokenHash	char(64)		UK		SHA-256 (hex) of the refresh token; the token itself is never stored.
ExpiresAt	datetime2				Expiry (UTC); default lifetime 7 days (Jwt:RefreshTokenDays).
CreatedAt	datetime2			SYSUTCDATETIME()	Issue time (UTC).
RevokedAt	datetime2	NULL			Set when the token is rotated, on sign-out or when the account is locked.
ReplacedByTokenId	uniqueidentifier	NULL			Token issued when this one was rotated; reuse of a revoked token revokes the whole chain.

Keys: PK_RefreshTokens (RefreshTokenId)

Unique indexes: UX_RefreshTokens_TokenHash (TokenHash)

Indexes: IX_RefreshTokens_UserId (UserId)

Foreign keys: FK_RefreshTokens_Users (UserId) -> Users (UserId), ON DELETE CASCADE

9.2 GroupDb (Group Service)

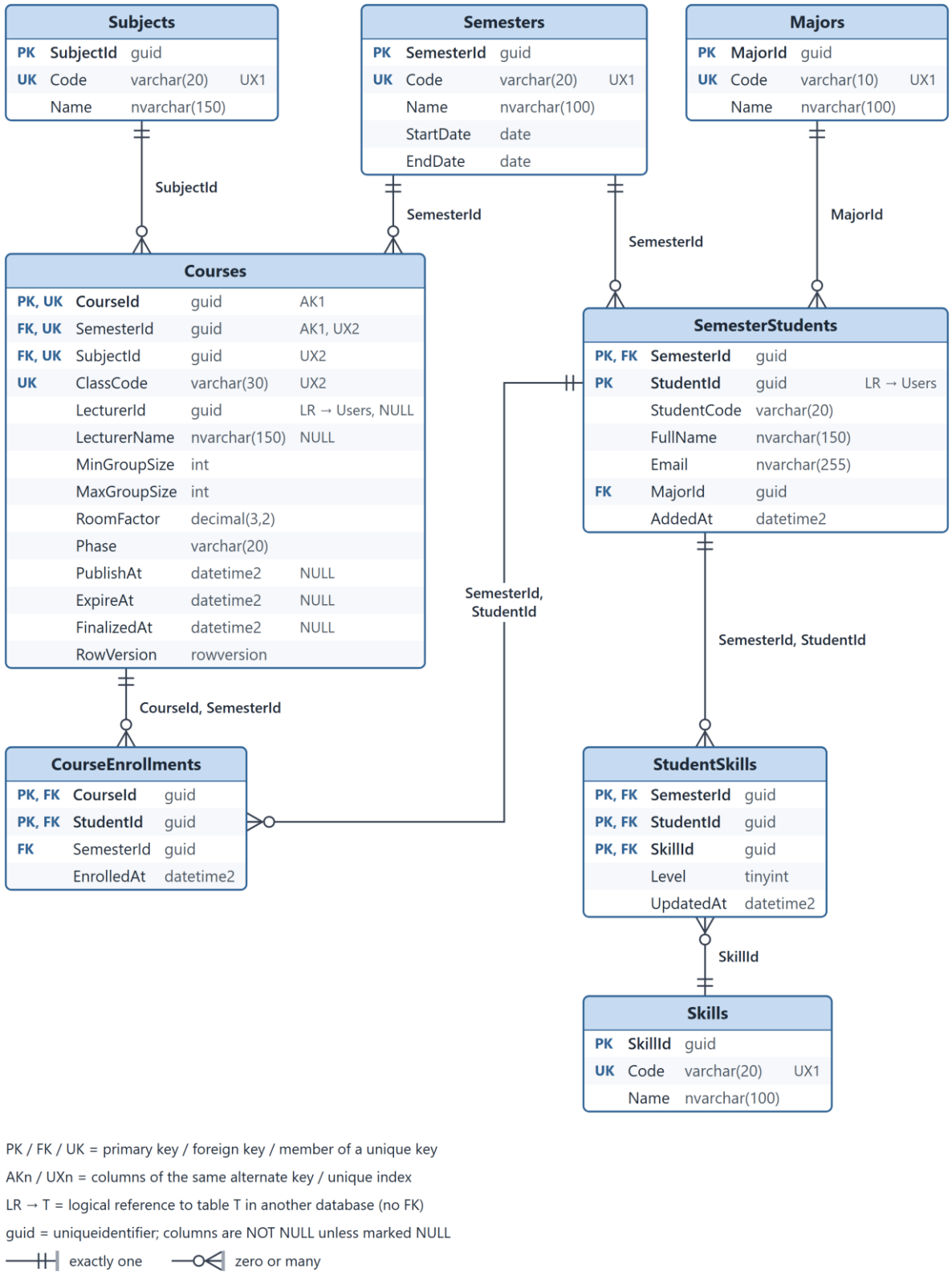


Figure 5a. Physical database: GroupDb (1/2), semesters, courses and students

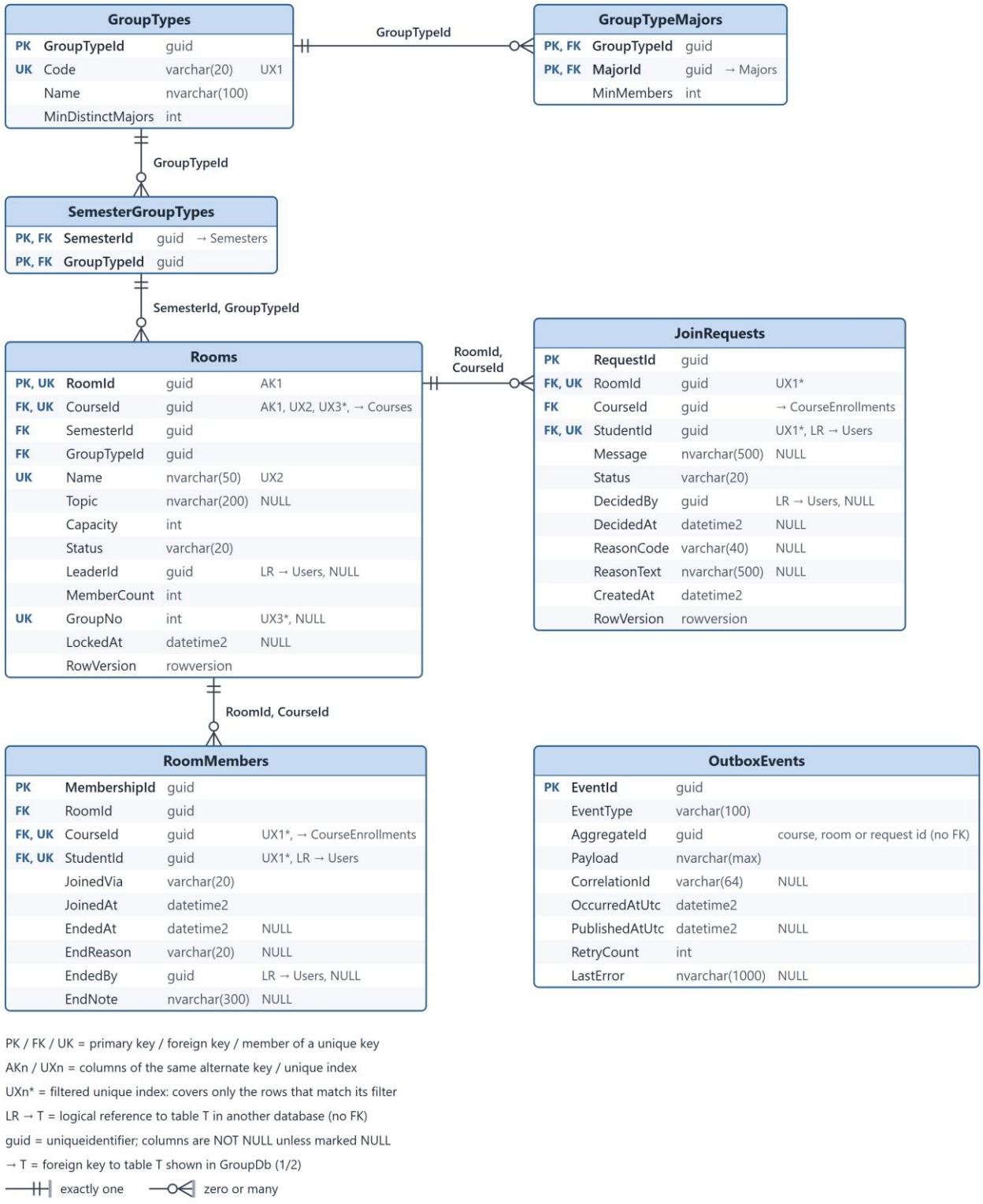


Figure 5b. Physical database: GroupDb (2/2), group types, rooms and applications

Semesters

Academic semesters.

Column	Type	Null	Key	Default	Description
SemesterId	uniqueidentifier		PK		Surrogate key.
Code	varchar(20)		UK		Semester code, e.g. FA26.
Name	nvarchar(100)				Display name, e.g. Fall 2026.
StartDate	date				First day of the semester.
EndDate	date				Last day; after StartDate.

Keys: PK_Semesters (SemesterId)

Unique indexes: UX_Semesters_Code (Code)

Checks: CK_Semesters_Dates: [EndDate] > [StartDate]

Subjects

Subjects taught across semesters, e.g. EXE101, EXE201.

Column	Type	Null	Key	Default	Description
SubjectId	uniqueidentifier		PK		Surrogate key.
Code	varchar(20)		UK		Subject code, e.g. EXE101.
Name	nvarchar(150)				Subject name.

Keys: PK_Subjects (SubjectId)

Unique indexes: UX_Subjects_Code (Code)

Majors

Study majors, e.g. SE, GD, MKT.

Column	Type	Null	Key	Default	Description
MajorId	uniqueidentifier		PK		Surrogate key.
Code	varchar(10)		UK		Major code, e.g. SE.
Name	nvarchar(100)				Major name.

Keys: PK_Majors (MajorId)

Unique indexes: UX_Majors_Code (Code)

Skills

Strengths students rate themselves on, e.g. Programming, Design, Marketing, Management.

Column	Type	Null	Key	Default	Description
SkillId	uniqueidentifier		PK		Surrogate key.
Code	varchar(20)		UK		Skill code.
Name	nvarchar(100)				Skill name.

Keys: PK_Skills (SkillId)

Unique indexes: UX_Skills_Code (Code)

SemesterStudents

Students taking part in a semester, with their major for that semester (imported by the admin).

Column	Type	Null	Key	Default	Description
SemesterId	uniqueidentifier		PK, FK		References Semesters.
StudentId	uniqueidentifier		PK, LR		UserId of the student in IdentityDb.
StudentCode	varchar(20)				Copied from IdentityDb at import, for lists and search.
FullName	nvarchar(150)				Copied from IdentityDb at import.
Email	nvarchar(255)				Copied from IdentityDb at import.
MajorId	uniqueidentifier		FK		Major of the student in this semester.
AddedAt	datetime2			SYSUTCDATETIME()	Import time (UTC).

Keys: PK_SemesterStudents (SemesterId, StudentId)

Indexes: IX_SemesterStudents_MajorId (MajorId)

Foreign keys: FK_SemesterStudents_Semesters (SemesterId) -> Semesters (SemesterId), ON DELETE NO ACTION; FK_SemesterStudents_Majors (MajorId) -> Majors (MajorId), ON DELETE NO ACTION

Logical references (no FK): StudentId -> Users

StudentSkills

Self-rated strengths (1-5) of a student in a semester.

Column	Type	Null	Key	Default	Description
SemesterId	uniqueidentifier		PK, FK		With StudentId: references SemesterStudents.
StudentId	uniqueidentifier		PK, FK		With SemesterId: references SemesterStudents.
SkillId	uniqueidentifier		PK, FK		References Skills.
Level	tinyint				Self-assessment 1..5.
UpdatedAt	datetime2			SYSUTCDATETIME()	Last change (UTC).

Keys: PK_StudentSkills (SemesterId, StudentId, SkillId)

Indexes: IX_StudentSkills_SkillId (SkillId)

Checks: CK_StudentSkills_Level: [Level] BETWEEN 1 AND 5

Foreign keys: FK_StudentSkills_SemesterStudents (SemesterId, StudentId) -> SemesterStudents (SemesterId, StudentId), ON DELETE NO ACTION; FK_StudentSkills_Skills (SkillId) -> Skills (SkillId), ON DELETE NO ACTION

Courses

One subject taught to one class in one semester (FAP course), with its grouping settings and phase.

Column	Type	Null	Key	Default	Description
CourseId	uniqueidentifier		PK, UK		Surrogate key.
SemesterId	uniqueidentifier		FK, UK		References Semesters.
SubjectId	uniqueidentifier		FK, UK		References Subjects.
ClassCode	varchar(30)		UK		Class, e.g. SE1919.
LecturerId	uniqueidentifier	NULL	LR		UserId of the lecturer (read-only access).
LecturerName	nvarchar(150)	NULL			Copied from IdentityDb for display.
MinGroupSize	int			4	Minimum members of an eligible room.
MaxGroupSize	int			6	Capacity of every generated room.
RoomFactor	decimal(3,2)			1.20	k in rooms = ceil(students / MaxGroupSize * k).
Phase	varchar(20)			'Setup'	Setup, Published, Expired, Finalized.
PublishAt	datetime2	NULL			Optional automatic publish time (UTC).
ExpireAt	datetime2	NULL			Grouping deadline (UTC); the course expires automatically.
FinalizedAt	datetime2	NULL			Set when the admin finalizes.
RowVersion	rowversion				Optimistic concurrency token.

Keys: PK_Courses (CourseId); AK_Courses_CourseId_SemesterId (CourseId, SemesterId)

Unique indexes: UX_Courses_Semester_Subject_Class (SemesterId, SubjectId, ClassCode)

Indexes: IX_Courses_SubjectId (SubjectId); IX_Courses_LecturerId (LecturerId) WHERE [LecturerId] IS NOT NULL; IX_Courses_Phase (Phase)

Checks: CK_Courses_GroupSize: [MinGroupSize] >= 1 AND [MaxGroupSize] >= [MinGroupSize]; CK_Courses_RoomFactor: [RoomFactor] >= 1.00; CK_Courses_Phase: [Phase] IN ('Setup', 'Published', 'Expired', 'Finalized'); CK_Courses_Schedule: [PublishAt] IS NULL OR [ExpireAt] IS NULL OR [ExpireAt] > [PublishAt]

Foreign keys: FK_Courses_Semesters (SemesterId) -> Semesters (SemesterId), ON DELETE NO ACTION; FK_Courses_Subjects (SubjectId) -> Subjects (SubjectId), ON DELETE NO ACTION

Logical references (no FK): LecturerId -> Users

CourseEnrollments

Students enrolled in a course (class roster).

Column	Type	Null	Key	Default	Description
CourseId	uniqueidentifier		PK, FK		With SemesterId: references Courses.
StudentId	uniqueidentifier		PK, FK		With SemesterId: references SemesterStudents.
SemesterId	uniqueidentifier		FK		Keeps the course and the student in the same semester.
EnrolledAt	datetime2			SYSUTCDATETIME()	Enrolment time (UTC).

Keys: PK_CourseEnrollments (CourseId, StudentId)

Indexes: IX_CourseEnrollments_Semester_Student (SemesterId, StudentId)

Foreign keys: FK_CourseEnrollments_Courses (CourseId, SemesterId) -> Courses (CourseId, SemesterId), ON DELETE NO ACTION; FK_CourseEnrollments_SemesterStudents (SemesterId, StudentId) -> SemesterStudents (SemesterId, StudentId), ON DELETE NO ACTION

GroupTypes

Kinds of groups with their major mix rules.

Column	Type	Null	Key	Default	Description
GroupTypeId	uniqueidentifier		PK		Surrogate key.
Code	varchar(20)		UK		Group type code.
Name	nvarchar(100)				Group type name.
MinDistinctMajors	int			2	At least this many different majors per group.

Keys: PK_GroupTypes (GroupTypeId)

Unique indexes: UX_GroupTypes_Code (Code)

Checks: CK_GroupTypes_MinDistinctMajors: [MinDistinctMajors] >= 1

GroupTypeMajors

Minimum number of students of a major in a group of this type (n:n GroupTypes - Majors).

Column	Type	Null	Key	Default	Description
GroupTypeId	uniqueidentifier		PK, FK		References GroupTypes.
MajorId	uniqueidentifier		PK, FK		References Majors.
MinMembers	int				Minimum students of this major per group.

Keys: PK_GroupTypeMajors (GroupTypeId, MajorId)

Indexes: IX_GroupTypeMajors_MajorId (MajorId)

Checks: CK_GroupTypeMajors_MinMembers: [MinMembers] >= 1

Foreign keys: FK_GroupTypeMajors_GroupTypes (GroupTypeId) -> GroupTypes (GroupTypeId), ON DELETE NO ACTION; FK_GroupTypeMajors_Majors (MajorId) -> Majors (MajorId), ON DELETE NO ACTION

SemesterGroupTypes

Group types allowed in a semester.

Column	Type	Null	Key	Default	Description
SemesterId	uniqueidentifier		PK, FK		References Semesters.
GroupTypeId	uniqueidentifier		PK, FK		References GroupTypes.

Keys: PK_SemesterGroupTypes (SemesterId, GroupTypeId)

Indexes: IX_SemesterGroupTypes_GroupTypeId (GroupTypeId)

Foreign keys: FK_SemesterGroupTypes_Semesters (SemesterId) -> Semesters (SemesterId), ON DELETE NO ACTION; FK_SemesterGroupTypes_GroupTypes (GroupTypeId) -> GroupTypes (GroupTypeId), ON DELETE NO ACTION

Rooms

Admin-created slots of a course; a room becomes an official group when the course is finalized.

Column	Type	Null	Key	Default	Description
RoomId	uniqueidentifier		PK, UK		Surrogate key.
CourseId	uniqueidentifier		FK, UK		With SemesterId: references Courses.
SemesterId	uniqueidentifier		FK		Keeps the room's group type allowed in the course's semester.
GroupTypeId	uniqueidentifier		FK		With SemesterId: references SemesterGroupTypes.
Name	nvarchar(50)		UK		Room 01, Room 02 ...; unique in the course.
Topic	nvarchar(200)	NULL			Project idea, set by the leader to attract members.
Capacity	int				= course MaxGroupSize when the rooms are generated.
Status	varchar(20)			'Open'	Open, Locked, Finalized.
LeaderId	uniqueidentifier	NULL	LR		First member by default; NULL when the room is empty.
MemberCount	int			0	Current members (EndedAt IS NULL).
GroupNo	int	NULL	UK		Official group number, set at finalize.
LockedAt	datetime2	NULL			When the leader locked the room.
RowVersion	rowversion				Optimistic concurrency token.

Keys: PK_Rooms (RoomId); AK_Rooms_RoomId_CourseId (RoomId, CourseId)

Unique indexes: UX_Rooms_Course_Name (CourseId, Name); UX_Rooms_Course_GroupNo (CourseId, GroupNo) WHERE [GroupNo] IS NOT NULL

Indexes: IX_Rooms_Semester_GroupType (SemesterId, GroupTypeId)

Checks: CK_Rooms_Status: [Status] IN ('Open', 'Locked', 'Finalized'); CK_Rooms_MemberCount: [MemberCount] >= 0 AND [MemberCount] <= [Capacity]; CK_Rooms_Leader: ([MemberCount] = 0 AND [LeaderId] IS NULL) OR ([MemberCount] > 0 AND [LeaderId] IS NOT NULL)

Foreign keys: FK_Rooms_Courses (CourseId, SemesterId) -> Courses (CourseId, SemesterId), ON DELETE NO ACTION; FK_Rooms_SemesterGroupTypes (SemesterId, GroupTypeId) -> SemesterGroupTypes (SemesterId, GroupTypeId), ON DELETE NO ACTION

Logical references (no FK): LeaderId -> Users

RoomMembers

Room memberships, current and ended (left, removed by the leader, moved by the admin).

Column	Type	Null	Key	Default	Description
MembershipId	uniqueidentifier		PK		Surrogate key.
RoomId	uniqueidentifier		FK		With CourseId: references Rooms.
CourseId	uniqueidentifier		FK, UK		With StudentId: references CourseEnrollments.
StudentId	uniqueidentifier		FK, UK, LR		UserId of the student.
JoinedVia	varchar(20)				Open, Application, Admin.
JoinedAt	datetime2			SYSUTCDATETIME()	Join time (UTC).
EndedAt	datetime2	NULL			NULL while the student is a member.
EndReason	varchar(20)	NULL			Left, Removed (by the leader), Moved (by the admin).
EndedBy	uniqueidentifier	NULL	LR		Leader or admin who ended the membership.
EndNote	nvarchar(300)	NULL			Reason given by the leader; required when Removed.

Keys: PK_RoomMembers (MembershipId)

Unique indexes: UX_RoomMembers_Course_Student (CourseId, StudentId) WHERE [EndedAt] IS NULL

Indexes: IX_RoomMembers_Room_Student (RoomId, StudentId)

Checks: CK_RoomMembers_JoinedVia: [JoinedVia] IN ('Open', 'Application', 'Admin'); CK_RoomMembers_End: ([EndedAt] IS NULL AND [EndReason] IS NULL) OR ([EndedAt] IS NOT NULL AND [EndReason] IN ('Left', 'Removed', 'Moved')); CK_RoomMembers_RemovedNote: [EndReason] IS NULL OR [EndReason] <> 'Removed' OR [EndNote] IS NOT NULL

Foreign keys: FK_RoomMembers_Rooms (RoomId, CourseId) -> Rooms (RoomId, CourseId), ON DELETE NO ACTION; FK_RoomMembers_CourseEnrollments (CourseId, StudentId) -> CourseEnrollments (CourseId, StudentId), ON DELETE NO ACTION

Logical references (no FK): StudentId -> Users; EndedBy -> Users

JoinRequests

Applications of students to locked rooms.

Column	Type	Null	Key	Default	Description
RequestId	uniqueidentifier		PK		Surrogate key.
RoomId	uniqueidentifier		FK, UK		With CourseId: references Rooms.
CourseId	uniqueidentifier		FK		With StudentId: references CourseEnrollments.
StudentId	uniqueidentifier		FK, UK, LR		UserId of the applicant.
Message	nvarchar(500)	NULL			Note from the student to the leader.
Status	varchar(20)			'Pending'	Pending, Approved, Rejected, Cancelled.
DecidedBy	uniqueidentifier	NULL	LR		Leader; NULL for automatic decisions.
DecidedAt	datetime2	NULL			Decision time (UTC).
ReasonCode	varchar(40)	NULL			Why it was rejected or cancelled (see the reason codes).
ReasonText	nvarchar(500)	NULL			Optional text from the leader.
CreatedAt	datetime2			SYSUTCDATETIME()	Submission time (UTC).
RowVersion	rowversion				Optimistic concurrency token.

Keys: PK_JoinRequests (RequestId)

Unique indexes: UX_JoinRequests_Pending (RoomId, StudentId) WHERE [Status] = 'Pending'

Indexes: IX_JoinRequests_Course_Student_Status (CourseId, StudentId, Status); IX_JoinRequests_Room_Status (RoomId, Status)

Checks: CK_JoinRequests_Status: [Status] IN ('Pending', 'Approved', 'Rejected', 'Cancelled'); CK_JoinRequests_ReasonCode: [ReasonCode] IS NULL OR [ReasonCode] IN ('REJECTED_BY_LEADER', 'WITHDRAWN', 'ROOM_FULL', 'ROOM_REOPENED', 'COMPOSITION_RULE', 'JOINED_ANOTHER_ROOM', 'GROUPING_EXPIRED')

Foreign keys: FK_JoinRequests_Rooms (RoomId, CourseId) -> Rooms (RoomId, CourseId), ON DELETE NO ACTION; FK_JoinRequests_CourseEnrollments (CourseId, StudentId) -> CourseEnrollments (CourseId, StudentId), ON DELETE NO ACTION

Logical references (no FK): StudentId -> Users; DecidedBy -> Users

OutboxEvents

Transactional outbox: events written in the same transaction as the business change.

Column	Type	Null	Key	Default	Description
EventId	uniqueidentifier		PK		Event id; also the idempotency key of the consumers.
EventType	varchar(100)				CoursePhaseChanged, RoomMemberJoined, RoomMemberLeft, ...
AggregateId	uniqueidentifier				Course, room or join request the event is about (no FK).
Payload	nvarchar(max)				Event body as JSON.
CorrelationId	varchar(64)	NULL			X-Correlation-ID of the request that caused the event.
OccurredAtUtc	datetime2			SYSUTCDATETIME()	When the change was committed.
PublishedAtUtc	datetime2	NULL			Set by the OutboxPublisher after XADD.
RetryCount	int			0	Failed publish attempts.
LastError	nvarchar(1000)	NULL			Last publish error.

Keys: PK_OutboxEvents (EventId)

Indexes: IX_OutboxEvents_Unpublished (OccurredAtUtc) WHERE [PublishedAtUtc] IS NULL; IX_OutboxEvents_PublishedAtUtc (PublishedAtUtc) WHERE [PublishedAtUtc] IS NOT NULL

Checks: CK_OutboxEvents_Payload: ISJSON([Payload]) = 1

9.3 ChatDb (Realtime Service)

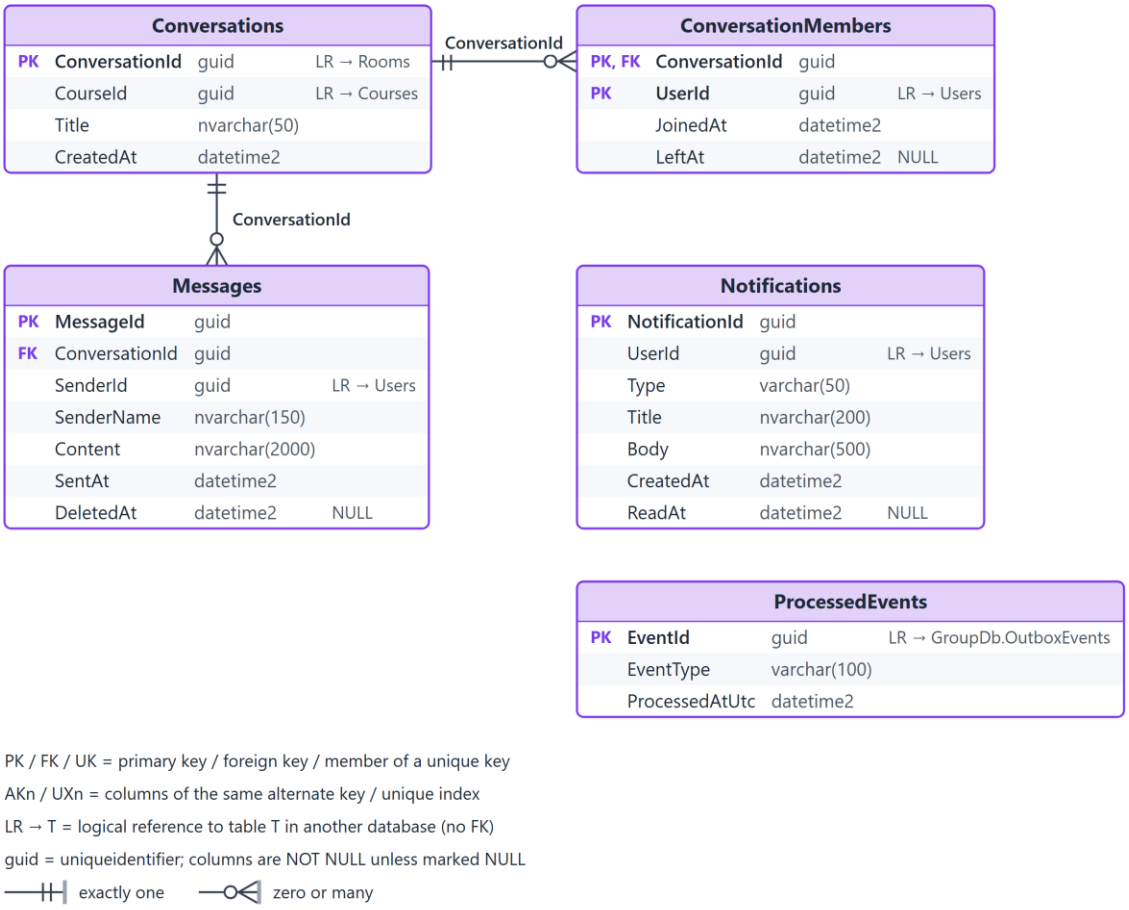


Figure 6. Physical database: ChatDb

Conversations

One conversation per room, created when the room gets its first member.

Column	Type	Null	Key	Default	Description
ConversationId	uniqueidentifier		PK, LR		= RoomId: one chat per room.
CourseId	uniqueidentifier		LR		Course of the room.
Title	nvarchar(50)				Room name snapshot.
CreatedAt	datetime2			SYSUTCDATETIME()	Creation time (UTC).

Keys: PK_Conversations (ConversationId)

Indexes: IX_Conversations_CourseId (CourseId)

Logical references (no FK): ConversationId -> Rooms; CourseId -> Courses

ConversationMembers

Who can read and write in a conversation, kept in sync from room events.

Column	Type	Null	Key	Default	Description
ConversationId	uniqueidentifier		PK, FK		References Conversations.

Column	Type	Null	Key	Default	Description
UserId	uniqueidentifier		PK, LR		Member (student) of the room.
JoinedAt	datetime2				From RoomMemberJoined.
LeftAt	datetime2	NULL			From RoomMemberLeft; the member keeps read access to older messages.

Keys: PK_ConversationMembers (ConversationId, UserId)

Indexes: IX_ConversationMembers_UserId (UserId)

Foreign keys: FK_ConversationMembers_Conversations (ConversationId) -> Conversations (ConversationId), ON DELETE NO ACTION

Logical references (no FK): UserId -> Users

Messages

Chat messages.

Column	Type	Null	Key	Default	Description
MessageId	uniqueidentifier		PK		Surrogate key.
ConversationId	uniqueidentifier		FK		References Conversations.
SenderId	uniqueidentifier		LR		UserId of the sender.
SenderName	nvarchar(150)				Name snapshot for display.
Content	nvarchar(2000)				Text of the message.
SentAt	datetime2			SYSUTCDATETIME()	Send time (UTC).
DeletedAt	datetime2	NULL			Set when the sender deletes the message.

Keys: PK_Messages (MessageId)

Indexes: IX_Messages_Conversation_SentAt (ConversationId, SentAt)

Foreign keys: FK_Messages_Conversations (ConversationId) -> Conversations (ConversationId), ON DELETE NO ACTION

Logical references (no FK): SenderId -> Users

Notifications

In-app notifications created from events.

Column	Type	Null	Key	Default	Description
NotificationId	uniqueidentifier		PK		Surrogate key.
UserId	uniqueidentifier		LR		Receiver.
Type	varchar(50)				Event that caused it, e.g. JoinRequestDecided.
Title	nvarchar(200)				Short title.
Body	nvarchar(500)				Text shown to the user.
CreatedAt	datetime2			SYSUTCDATETIME()	Creation time (UTC).
ReadAt	datetime2	NULL			Set when the user reads it.

Keys: PK_Notifications (NotificationId)

Indexes: IX_Notifications_Unread (UserId, CreatedAt) WHERE [ReadAt] IS NULL; IX_Notifications_User_CreatedAt (UserId, CreatedAt)

Logical references (no FK): UserId -> Users

ProcessedEvents

Inbox of consumed event ids (idempotent consumer).

Column	Type	Null	Key	Default	Description
EventId	uniqueidentifier		PK, LR		EventId of GroupDb.OutboxEvents.
EventType	varchar(100)				Type of the consumed event.
ProcessedAtUtc	datetime2			SYSUTCDATETIME()	Processing time (UTC).

Keys: PK_ProcessedEvents (EventId)

Indexes: IX_ProcessedEvents_ProcessedAtUtc (ProcessedAtUtc)

Logical references (no FK): EventId -> GroupDb.OutboxEvents

Appendix A. REST API

Every route is served by the gateway under /api/v1 (shown without the prefix) and answers with the envelope of section 6.8. List endpoints also accept search, sort, page, pageSize, fields and expand.

Identity Service

Method	Route	Roles	Purpose
POST	/auth/login	Anonymous	Sign in; returns an access token and a refresh token.
POST	/auth/refresh-token	Anonymous	Rotate the refresh token and get a new access token.
POST	/auth/logout	Signed in	Revoke the refresh token.
GET, PUT	/auth/me	Signed in	Read or update own name and password.
GET	/users	Administrator	List users: search name, e-mail or code; filter role, status; sort; page.
GET, PUT	/users/{id}	Administrator	Read or update a user: name, roles, status (lock revokes tokens).
POST	/users	Administrator	Create an account with its roles, e.g. a lecturer.
POST	/users/import	Administrator	Create or update student accounts in bulk (by e-mail); returns their ids.

Group Service: master data

Method	Route	Roles	Purpose
GET, POST	/semesters	Read: all; write: Administrator	List (search, sort, page) and create.
GET, PUT, DELETE	/semesters/{id}	Read: all; write: Administrator	Read, update, delete.
GET, POST, PUT, DELETE	/subjects, /majors, /skills (+ /{id})	Read: all; write: Administrator	Same CRUD and list conventions.
GET, POST, PUT, DELETE	/group-types (+ /{id})	Read: all; write: Administrator	Group types; PUT /group-types/{id}/majors sets the minimum per major.
GET, PUT	/semesters/{id}/group-types	Read: all; write: Administrator	Group types allowed in the semester.

Group Service: students and courses

Method	Route	Roles	Purpose
GET, POST	/semesters/{id}/students	Administrator	List (search, filter majorId, sort, page); add or update students with their major (bulk).
DELETE	/semesters/{id}/students/{studentId}	Administrator	Remove a student not enrolled anywhere.
GET	/semesters/{id}/students/me	Student	Own major and skills.
PUT	/semesters/{id}/students/me/skills	Student	Rate own skills 1-5.
GET, POST	/courses	Read: all (mine=true); write: Administrator	List (filter semesterId, subjectId, phase, mine; search class code; sort; page) and create.
GET, PUT, DELETE	/courses/{id}	Read: course members, lecturer, Administrator; write: Administrator	Read; update settings, lecturer, PublishAt and ExpireAt; delete in Setup.
GET, POST	/courses/{id}/students	Read: Administrator,	Roster (filter majorId, hasRoom; search;

Method	Route	Roles	Purpose
		lecturer; write: Administrator	sort; page); enroll semester students.
DELETE	/courses/{id}/students/{studentId}	Administrator	Remove a student without a room.
POST	/courses/{id}/rooms/generate	Administrator	Setup: create the rooms (BR-03).
POST	/courses/{id}/publish, /courses/{id}/expire	Administrator	Change the phase now.
GET	/courses/{id}/overview	Administrator, lecturer	Counts, students without a room, rooms that are not eligible.
GET	/courses/{id}/placement-suggestions	Administrator	Expired: suggestions (gRPC).
POST	/courses/{id}/placements	Administrator	Expired: move or place students.
POST	/courses/{id}/finalize	Administrator	Finalize (BR-14) with acceptExceptions.

Group Service: rooms and applications

Method	Route	Roles	Purpose
GET	/courses/{id}/rooms	Course members, lecturer, Administrator	Filter status, hasSeat, groupTypeId; search name or topic; sort name or memberCount; page.
GET	/courses/{id}/rooms/suggestions	Student	Top rooms with reasons (gRPC).
GET, PUT	/rooms/{id}	Read: course members, lecturer, Administrator; write: leader (topic), Administrator (Setup)	Room with members, majors, skills and leader.
DELETE	/rooms/{id}	Administrator	Delete an empty room in Setup.
POST	/rooms/{id}/join, /rooms/{id}/leave	Student	Join an open room; leave a room.
POST	/rooms/{id}/lock, /rooms/{id}/unlock	Room leader	Switch between Open and Locked.
POST	/rooms/{id}/leader	Room leader	Hand over leadership to a member.
POST	/rooms/{id}/members/{studentId}/remove	Room leader	Remove a member with a reason (BR-11).
GET, POST	/join-requests	Read: own, leader of the room, Administrator; write: Student	List (filter roomId, courseId, status; sort; page); apply to a locked room.
GET	/join-requests/{id}	Applicant, leader, Administrator	One application.
POST	/join-requests/{id}/approve, /reject, /withdraw	Leader (approve, reject); applicant (withdraw)	Decide or withdraw.

Realtime Service

Method	Route	Roles	Purpose
GET	/conversations	Signed in	Own room chats with the last message.
GET, POST	/conversations/{id}/messages	Room members	Messages, newest first (before, pageSize); send a message.
DELETE	/messages/{id}	Sender	Delete own message.
GET	/notifications	Signed in	Own notifications (filter unread; page).
POST	/notifications/{id}/read, /notifications/read-all	Signed in	Mark as read.
WebSocket	/hubs/realtime	Signed in	Server to client: notificationReceived, messageReceived, roomChanged.

Appendix B. Assignment requirements

Requirement (assignment brief)	Where it is met
2.1 REST API service	Identity, Group and Realtime services (Appendix A); CRUD for semesters, subjects, majors, skills, group types, courses, rooms and users; layered architecture and JWT (section 6.8); search, filters, sorting and paging on every list.
2.2 Background job	PhaseScheduler, OutboxPublisher, clean-ups and the GroupEventsConsumer (section 6.6).
2.3 Message broker	Redis Streams: the Group Service produces, the Realtime Service consumes (section 6.5).
2.4 gRPC service	Recommendation Service: independent, gRPC only, called by the Group Service REST API

Requirement (assignment brief)	Where it is met
	(sections 6.4 and 6.7).
3 Deployment	Docker Compose with all containers; demo server deployed by GitHub Actions.
4 Technical requirements	.NET 10, EF Core 10, SQL Server 2022, dependency injection, configuration, Serilog and exception middleware, JWT, REST, gRPC, Redis, BackgroundService, Docker (section 7).
5 Deliverables	Source code, EF Core migrations and database/schema.sql, Dockerfiles, docker-compose.yml, Swagger at the gateway, README.md.
6 Demonstration	README demo script: setup, publish, join, apply and approve, suggestions, expiry by the job, placement, finalize, chat.